

Nome: _____ Matrícula: _____

Instruções.

- São 18 questões. **Escolha as questões que quiser responder**; o ideal é que a soma dos pontos seja *exatamente* 10,0. Indique, no início da prova, as questões escolhidas.
- **Se a soma das pontuações das questões entregues exceder 10,0**, a nota será **normalizada** pela soma total entregue: $\text{nota} = 10,0 \times \frac{\text{pontos obtidos}}{\text{soma das pontuações das questões entregues}}$.
- Resolva à **mão**, em folha própria. Ao final, **escaneie** ou fotografe e **converta para um único PDF**, entregando no SIGAA. Páginas ilegíveis serão desconsideradas.
- Escreva o seu **nome e matrícula em todas as folhas**.
- **Estilo funcional obrigatório**: use `map`, `filter`, `reduce` (de `functools`), **comprehensions** e/ou **recursão**. **Proibido**: laços `for/while`, `.append`, mutação por índice, classes, POO. Violar zera a questão. **Atenção**: quando o enunciado pedir explicitamente `map/filter/reduce` ou **recursão**, use a técnica pedida — uma **compreensão** não substitui nesses casos.
- Como `map/filter` devolvem iteradores, **converta o resultado para lista** com `list(...)` quando a questão pedir uma lista (ex.: `list(map(...))`); `list()`, `tuple()` e literais de `dict/tupla` são **permitidos** e não contam como laço.
- Pode usar `from functools import reduce` e `import math` livremente.

Questão 1 — Variação real do PIB (1,0 pts)

O PIB nominal brasileiro (em R\$ bilhões) dos últimos quatro trimestres foi:

$$pib = [2780, 2815, 2860, 2902]$$

Escreva, em **uma única expressão** com `map`, a função `variacoes(pib)` que retorne a lista das variações percentuais trimestre-a-trimestre (em %), arredondadas a 2 casas decimais. *Dica*: `map` aceita duas listas ao mesmo tempo (ex.: `map(f, pib[:-1], pib[1:])`). Exemplo de saída esperada (aproximada, separada por “;” por causa da vírgula decimal): `[1,26; 1,60; 1,47]`.

Apresente **2 asserts** de teste.

Questão 2 — Filtrando trimestres de alta inflação (1,0 pts)

Dada uma lista `ipca` contendo a inflação mensal medida pelo IPCA (em %), defina `meses_acima_da_meta(ipca, meta)` usando **somente** `filter` (envolvido por `list`), que devolve a lista dos valores estritamente acima de `meta`.

Em seguida, defina `qtd_acima(ipca, meta)` usando `filter` + `reduce` (sem `len`).

Exemplo:

```
ipca = [0.21, 0.56, 0.83, 0.38, 0.71, 0.49]
assert meses_acima_da_meta(ipca, 0.50) == [0.56, 0.83, 0.71]
assert qtd_acima(ipca, 0.50) == 3
```

Questão 3 — Selic acumulada com reduce (1,0 pts)

A taxa Selic mensal (em fração, ex.: 0,01 para 1%) de seis meses é dada em `selic_mensal`. Defina, em **uma única expressão** usando `reduce`, a função `selic_acumulada(selic_mensal)` que devolve a **taxa acumulada (equivalente) do período** $\prod_i (1 + r_i) - 1$.

```
selic_mensal = [0.0090, 0.0092, 0.0095, 0.0093, 0.0091, 0.0090]
# selic_acumulada(selic_mensal) ~ 0.0564
```

Justifique em 1-2 linhas o porquê do `-1` no final.

Questão 4 — Deflacionar valores nominais (1,0 pts)

Dada uma lista `salarios` (valores nominais em R\$) e uma inflação acumulada `infl` (em fração, ex.: 0,045

para 4,5%), escreva, usando **apenas** `map`, a função `deflacionar(salarios, infl)` que retorne os salários em moeda **real** (i.e., divididos por $1 + infl$), arredondados a 2 casas.

Apresente 2 asserts.

Questão 5 — Aquecimento funcional

(1,0 pts)

Considere países representados por tuplas `(nome, pib)`, com o PIB em **bilhões de US\$**. Reescreva o trecho **imperativo** abaixo, que calcula o PIB total dos países com PIB acima de US\$ 1 trilhão (i.e., `pib > 1000`), em **uma única expressão** usando `filter` + `reduce` (sem laços, sem variáveis intermediárias):

```
def pib_grandes(paises):
    total = 0
    for nome, pib in paises:
        if pib > 1000:
            total = total + pib
    return total
```

Inclua 2 asserts com uma lista exemplo de seu gosto.

Questão 6 — Juros compostos por recursão

(1,5 pts)

Sem usar laços, `**` ou `math.pow`, defina **recursivamente** a função `montante(c, i, n)` que devolve $C \cdot (1+i)^n$, onde C é o capital, i a taxa mensal e $n \geq 0$ o número de meses inteiros.

- (a) Defina explicitamente o **caso-base** e o **caso recursivo**, justificando a terminação.
 - (b) Mostre que `montante(1000, 0.01, 3) = 1030.301`.
 - (c) Apresente 2 asserts.
-

Questão 7 — Recursão sobre naturais: dígitos do PIB

(1,5 pts)

Defina **recursivamente** (sem laços, sem `str`, sem `map/filter/reduce`):

- (a) `soma_digitos(n)` — soma dos dígitos de $n \geq 0$. *Ex.*: `soma_digitos(2902) = 13`.
- (b) `raiz_digital(n)` — aplica `soma_digitos` repetidamente **de forma recursiva**, até sobrar um único dígito. *Ex.*: `raiz_digital(2902) = 4`.

Justifique a terminação de `raiz_digital`.

Questão 8 — Recursão estrutural sobre listas de IPCA

(1,5 pts)

Implemente, **apenas por recursão estrutural** (com cabeça `xs[0]` e cauda `xs[1:]`; sem `map/filter/reduce`, sem laços, sem `len`, `sum`, `max`):

- (a) `maior_ipca(xs)` — maior valor da lista (assuma `xs` não-vazia);
- (b) `soma_ipca(xs)` — soma da lista;
- (c) `media_ipca(xs)` — média aritmética, usando (a) e (b) e uma função auxiliar `tamanho(xs)` também recursiva.

Apresente 2 asserts para cada item.

Questão 9 — Boletim macroeconômico (pipeline)

(2,0 pts)

Uma consultoria mantém indicadores de países no formato `(pais, pib_per_capita, inflacao)`:

```
dados = [
    ("Brasil", 10500, 4.5),
    ("Argentina", 9800, 80.0),
    ("Chile", 17200, 3.6),
    ("Peru", 7900, 5.1),
    ("Colombia", 8400, 6.2),
    ("Uruguai", 22500, 7.8),
]
```

Diremos que um país é **ESTAVEL** quando sua inflação for $\leq 6,0\%$, e **INSTAVEL** caso contrário. Implemente, **cada uma em uma única expressão** com `map/filter/reduce` (sem laços, sem comprehensions):

- (a) `estaveis(dados)` — nomes dos países ESTAVEIS (inflação $\leq 6,0\%$);
- (b) `pib_medio(dados)` — média do PIB per capita, usando `reduce` (sem `sum`);
- (c) `relatorio(dados)` — lista de strings no formato "Brasil: PIB_pc=10500 INFL=4.5% (ESTAVEL)" ou "... (INSTAVEL)" (usando o critério acima), com `map`;
- (d) `mais_rico(dados)` — tupla (`pais`, `pib_per_capita`) com o *maior* PIB per capita, via `reduce` (sem `max`). Apresente 2 asserts para cada item.

Questão 10 — Implementando minha_reduce

(2,0 pts)

Sem usar `map`, `filter`, `reduce` ou laços, defina **por recursão** sobre listas:

```
def minha_reduce(f, xs, inicial):  
    """Equivalente a functools.reduce(f, xs, inicial)."""
```

Em seguida, usando **somente** `minha_reduce` e `lambda`, escreva **em uma única linha** cada:

- (a) `soma_pib(xs)` — soma dos PIBs em `xs`;
- (b) `maior_juros(xs)` — maior valor de `xs` (sem `max`; assumo `xs` não-vazia, use `xs[0]` como inicial);
- (c) `inverte(xs)` — lista invertida;
- (d) `meu_map(f, xs)` — equivalente a `list(map(f, xs))`.

Questão 11 — Inflação acumulada por recursão e por reduce

(2,0 pts)

Dada uma lista `infl` com inflações mensais em fração, a inflação acumulada do período é

$$I = \prod_{k=1}^n (1 + i_k) - 1.$$

- (a) Implemente `acumulada_rec(infl)` **por recursão estrutural** (cabeça/cauda), sem laços e sem `reduce`.
- (b) Implemente `acumulada_red(infl)` em **uma única expressão** com `reduce`.
- (c) Demonstre, com 3 asserts, que ambas concordam (use `math.isclose`) para listas vazias, unitárias e de tamanho ≥ 3 .
- (d) Justifique o caso-base de (a) e por que `acumulada_rec([])` deve retornar 0.

Questão 12 — Achatar séries temporais aninhadas

(2,0 pts)

Um sistema de coleta agrupa o IPCA por ano e por trimestre, produzindo listas **aninhadas** em profundidade arbitrária. Meses ainda **não medidos** são registrados como 0.0. Por exemplo:

`[[0,21; [0,56; 0,83]]; []; [[0,38]; 0,0; 0,71].`

- (a) Implemente `achatar(xs)` por **recursão estrutural** (a cabeça pode ser uma lista). Use `isinstance(x, list)` para distinguir.
- (b) Usando `achatar` + `filter` + `reduce` em uma única expressão, defina `ipca_anual(xs)` que retorna $\prod (1 + i_k) - 1$ **considerando apenas** os meses efetivamente medidos (descarte as medições iguais a 0.0, que sinalizam ausência de medição; meses de deflação, com valor negativo, *devem* entrar).
- (c) Apresente 2 asserts para cada item.

Questão 13 — Tempo para dobrar o capital (recursão)

(2,5 pts)

Dado um capital aplicado à taxa mensal i (fração), o **tempo de duplicação** é o menor $n \in \mathbb{N}$ tal que $(1+i)^n \geq 2$.

- (a) Defina, **por recursão** (sem laços, sem `math.log`, sem `**`), a função `meses_para_dobrar(i)`. *Dica:* use uma auxiliar recursiva que mantém o fator acumulado.
- (b) Prove informalmente a terminação para $i > 0$.

- (c) Calcule, à mão, `meses_para_dobrar(0.5)` e `meses_para_dobrar(0.25)`, mostrando as chamadas recursivas até o caso-base (pare quando o fator atingir ou passar de 2).
- (d) Apresente 3 asserts, incluindo um caso de borda que você considere relevante. Justifique sua escolha.
-

Questão 14 — Ranking de países por estabilidade

(2,5 pts)

Recebe-se uma lista `paises` de tuplas (`nome`, `[ipca_mensal_12m]`), isto é, cada país tem seu vetor com 12 inflações mensais (em fração).

- (a) Usando `map + reduce`, defina `inflacao_anual(paises)` que devolve a lista de tuplas (`nome`, `inflacao_acumulada`), onde a inflação acumulada de cada país é $\prod_k (1 + i_k) - 1$.
- (b) Usando `filter`, defina `cumprem_meta(paises, meta)` que retorna apenas os países cuja inflação acumulada seja $\leq meta$.
- (c) Usando `reduce` (sem `min`, sem `sorted`), defina `mais_estavel(paises)` que retorna a tupla (`nome`, `infl`) com **menor** inflação acumulada.
- (d) Combine (a), (b) e (c) em uma função `relatorio_estabilidade(paises, meta)` que devolve um dicionário com chaves `"cumprem"` (lista de (a) filtrada por (b)), `"mais_estavel"` (resultado de (c)) e `"qtd"` (quantidade de países que cumprem a meta). Apresente 2 asserts com dados de sua escolha.
-

Questão 15 — Modelo dinâmico do PIB (recursão + funcional)

(3,0 pts)

Suponha um modelo dinâmico simplificado em que o PIB de cada período depende do PIB anterior, do crescimento base e da taxa de juros:

$$pib_{t+1} = pib_t \cdot (1 + g) - \beta \cdot j_t,$$

onde g é a taxa base de crescimento, $\beta > 0$ a sensibilidade ao juros e j_t a taxa de juros do período t (lista `juros` de tamanho T).

- (a) Defina **por recursão estrutural** sobre a lista `juros`, a função `trajetoria(pib0, g, beta, juros)` que devolve a lista `[pib0, pib1, ..., pibT]` (com $T + 1$ elementos). Não use laços nem mutação. Não use `reduce`.
- (b) Reescreva (a) usando `reduce`, devolvendo a mesma lista. *Dica:* o acumulador é a lista de PIBs já calculados; combine com o `juros` corrente.
- (c) Usando o resultado de (a) ou (b) e `map/filter/reduce`, defina:
- `pib_medio_trajetoria(...)` — média dos PIBs da trajetória;
 - `periodos_recessao(...)` — quantos períodos tiveram $pib_{t+1} < pib_t$.
- (d) Com $pib_0 = 100$, $g = 0,02$, $\beta = 50$, $juros = [0,01; 0,03; 0,05; 0,02]$, calcule à mão a trajetória completa (resposta esperada: `[100; 101,5; 102,03; 101,5706; 102,602]`) e verifique que sua função (a) produz o mesmo resultado.
- (e) Justifique formalmente a terminação da recursão em (a).
-

Questão 16 — Cesta de preços (list comprehension)

(1,5 pts)

Uma cesta de produtos é dada como tuplas (`produto`, `preco_ant`, `preco_atual`):

```
cesta = [("arroz", 5.20, 5.62), ("feijao", 8.90, 9.40),
         ("leite", 4.50, 4.41), ("cafe", 18.0, 22.5)]
```

Usando **list comprehensions** (uma em cada item; sem `map/filter`, sem laços):

- (a) `variacoes(cesta)` — lista de tuplas (`produto`, `var_%`), com a variação percentual arredondada a 2 casas. *Ex.:* `("cafe", 25.0)` e `("leite", -2.0)`.
- (b) `encareceram(cesta, limite)` — lista dos **nomes** dos produtos cuja variação foi *estritamente acima* de `limite%` (use a condição `if` da própria `comprehension`). *Ex.:* `encareceram(cesta, 10.0) == ["cafe"]`.
- Apresente 2 asserts para cada item.
-

Questão 17 — Estabilidade entre países (set comprehension)

(1,5 pts)

Observações anuais de inflação no formato (`pais`, `ano`, `inflacao_%`) (um país aparece em vários anos):

```
obs = [("Brasil", 2024, 4.5), ("Brasil", 2025, 6.8),
       ("Chile", 2024, 3.6), ("Chile", 2025, 3.9),
       ("Peru", 2024, 5.1), ("Peru", 2025, 4.8),
       ("Argentina", 2024, 80.0), ("Argentina", 2025, 120.0)]
```

Usando **set comprehensions** e operações de conjunto (sem `map/filter`, sem laços):

- (a) `paises(obs)` — **conjunto** dos países distintos presentes em `obs`.
- (b) `instaveis(obs, meta)` — conjunto dos países que em **algum** ano tiveram inflação $>$ `meta`.
- (c) `sempre_estaveis(obs, meta)` — conjunto dos países que **nunca** ultrapassaram `meta`, obtido por **diferença de conjuntos** entre (a) e (b). *Ex.:* com `meta = 6.0`, `sempre_estaveis(obs, 6.0) == {"Chile", "Peru"}`.

Explique em 1 linha por que o conjunto evita contar o mesmo país duas vezes. Apresente 2 asserts para cada item.

Questão 18 — Carteira de empréstimos (lista + conjunto) (2,0 pts)

Taxas mensais de empréstimo no formato (`banco`, `modalidade`, `taxa_mensal`) (em fração):

```
emp = [("BancoA", "PME", 0.012), ("BancoB", "PME", 0.015),
       ("BancoA", "Habitacao", 0.008), ("BancoC", "Consignado", 0.009),
       ("BancoB", "Consignado", 0.011), ("BancoA", "PME", 0.013)]
```

- (a) Com uma **list comprehension**, defina `anualizar(emp)` — lista de (`banco`, `modalidade`, `taxa_anual`), onde `taxa_anual = round((1+taxa_mensal)**12 - 1, 4)`. *Ex.:* a primeira tupla vira `("BancoA", "PME", 0.1539)`.
- (b) Com uma **set comprehension**, defina `modalidades(emp)` — conjunto das modalidades distintas oferecidas.
- (c) Com uma **set comprehension** (com condição), defina `bancos_caros(emp, limite)` — conjunto dos bancos que oferecem **alguma** linha com taxa mensal $>$ `limite`. *Ex.:* `bancos_caros(emp, 0.011) == {"BancoA", "BancoB"}`.

Apresente 2 asserts para cada item.

Combinações sugeridas para somar 10,0: $\{15, 13, 14\} = 3+2,5+2,5 = 8,0$ (*não chega a 10!*). Exemplos válidos: $\{15, 13, 14, 9\} = 3+2,5+2,5+2 = 10,0$; $\{13, 14, 9, 6, 7\} = 2,5+2,5+2+1,5+1,5 = 10,0$; $\{9, 10, 11, 12, 1, 2\} = 2+2+2+2+1+1 = 10,0$. Escolha a sua: o ideal é somar 10,0; se a soma entregue exceder 10,0, a nota é normalizada (veja as instruções).